



Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Environment
Actuators
Sensors



2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.food_positions` — where food is placed
`self.opponents` — adversary positions
`self.agent_pos` — agent's location in the world



3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent. The `self.opponents` list contains independently moving agents that act every step and can collide with the player. Since there are multiple active decision-making entities in the same environment, it's Multi-Agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the full history of all percepts the agent has received from the start up to the current moment — basically everything the agent has ever sensed, in order.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S). It's how the agent reads the environment — like eyes giving it a snapshot of what's happening right now.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable. The agent can see its own position, opponent positions, and a few boolean flags — but it cannot see the exact locations of all remaining food pellets or traps. It only knows how many food pellets are left, not where they are.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P). `self.score -= 5` changes the metric used to judge how well the agent is doing.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Because the whole point of the agent is to produce good outcomes in the world. If you measure internal effort (like how many calculations it did), the agent could just spin in loops and score perfectly without doing anything useful. Measuring wall hits ties the score directly to



Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It makes the environment more partially observable. The traps exist in the world but the agent has no way of sensing them — so it could step on one and get penalised without any warning. The agent's view of the world becomes less complete, increasing uncertainty.



Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

A rational agent tries to maximise its score. If `smells_toxin = True`, the agent now knows it's on a dangerous cell and can choose to move away instead of staying and losing 15 points every step. Without this sensor, the agent is flying blind. With it, it can make a smarter decision —



Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The vacuum agent was rewarded every time it cleaned a dirty square. The exploit was that it could dirty a clean square and immediately clean it again — over and over — to earn infinite points without actually keeping the environment clean. The lesson: if your performance metric



Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING